

二分图增广路算法：从 +1 改造到交替可达集合

一份保留思维连续性的长版推导笔记

写作原则

这不是从标准答案倒推的算法讲义，而是按“问题意识 -> 暴力想法 -> 反例/爆炸 -> 状态压缩 -> BFS 形式”的顺序整理的一份推导过程笔记。

目录

- 1. 为什么先找 +1，而不是直接找最大？
- 2. “+1 可行性检验”和 FPT 思维的相似与不同
- 3. 暴力枚举增广路：为什么像在撞 NP-hard 的墙
- 4. 从边到点：为什么边集合/路径集合会爆炸
- 5. 关键转向：路径历史的等价关系
- 6. 多源初始左点集合
- 7. 右点扩展、左点回传，以及闭包稳定
- 8. 算法不变量：为什么可以忘掉路径历史
- 9. 终止条件和正确性直觉
- 10. 复杂度直觉
- 11. 这次推导真正训练到的东西

1. 为什么先找 +1，而不是直接找最大？

最大匹配的目标看起来是一个全局优化问题：在所有互不共享端点的边集合里，找出数量最大的那一个。直接从“最大”出发，很容易让人想到枚举组合：每条边要不要选、每个点要和谁匹配、当前匹配边是否应该被替换。这个角度很快变成指数级。

一个更自然的算法设计问题是：既然 matching 的大小是整数，那么从当前 matching M 出发，最小的正向改进单位是什么？答案就是 +1。也就是说，不必一口气问“最大在哪里”，可以先问：

给定当前 matching M ，是否存在一种局部改造，使 $|M|$ 变成 $|M| + 1$ ？

这个问题比原问题小得多。它把“最大化”转成了“可行性检验”：当前状态是否还能改进一格。如果能，就改；如果不能，并且我们能证明不存在任何 +1 改造，那么当前 matching 就已经是最大。

核心思想

“找最大”被拆成一串“能不能 +1”的问题。+1 不是技巧，而是 matching 这种离散结构的最小可观测改进单位。

从这个角度看，增广路不是凭空出现的定义，而是从“怎样让匹配数净增加 1”这个 accounting 问题自然推出来的。假设我们沿某个结构翻转边：原来不在 M 的边加入，原来在 M 的边删除。如果加入的边数比删除的边数多 1，并且翻转后仍然满足每个点最多被匹配一次，那么 matching 的大小就 +1。

新增边数 - 删除边数 = 1

并且翻转后仍是合法 matching

=> 匹配数增加 1

2. “+1 可行性检验”和 FPT 思维的相似与不同

你提到 FPT (fixed-parameter tractable) 的类比, 这个类比在“算法设计范式”层面是有价值的。很多 FPT 问题不是直接问最优值, 而是把问题改写成带参数的可行性问题。例如: 是否存在大小不超过 k 的 vertex cover? 是否存在包含 k 个对象的结构? 这种问法把优化问题切成一系列“给定目标规模, 是否可行”的判断。

二分图 matching 这里的 +1 过程有类似味道: 当前大小是 $|M|$, 我们问是否能做到 $|M|+1$ 。能做到, 就更新; 做不到, 就停。它们共同体现了一种重要思维:

优化问题

=> 局部/参数化的可行性问题

=> 找一个证书

=> 若证书存在则推进状态

=> 若证书不存在则证明当前状态无法继续改进

但严格复杂度意义上, 二分图 matching 不是 FPT 问题。FPT 通常强调复杂度形如 $f(k) * \text{poly}(n)$, 指数部分只依赖参数 k 。matching 的增广路算法更强: 它最终不是靠“小参数”控制搜索, 而是靠结构压缩直接得到多项式时间。

比较项	FPT 典型思路	二分图增广路思路
基本问法	给定参数 k , 是否存在规模为 k 的结构?	给定当前 M , 是否存在让 $ M +1$ 的结构?
搜索对象	可能是大小 k 的组合证书	一条交替路径形式的 +1 证书
复杂度来源	指数部分通常依赖 k	若枚举路径会指数爆炸, 但可压成点集合可达性
关键突破	分支规则、kernel、参数化状态	证明路径历史可被交替可达状态替代
最终性质	fixed-parameter tractable	普通多项式时间

所以你的类比应当这样表述: 不是说增广路算法是 FPT, 而是说它的早期设计动作——把全局优化拆成 +1 可行性检验——和 FPT/参数化搜索的思维方式有共同内核。不同点在于: 这里的结构更好, 最后能从“证书搜索”压成普通多项式 BFS。

3. 从 +1 出发推增广路: 数量 accounting

假设当前 matching 是 M 。我们想做一次改造, 改造后仍然是 matching, 并且数量多 1。最自然的局部改造是沿一条路径翻转: 路径上的非匹配边变成匹配边, 匹配边变成非匹配边。

为什么路径边数必须是奇数? 因为如果一条路径从未匹配左点出发, 到未匹配右点结束, 并且边类型按“非匹配、匹配、非匹配、匹配、……、非匹配”交替, 那么非匹配边比匹配边多一条。翻转以后就净增一条匹配边。

非匹配边数量 = 匹配边数量 + 1

翻转后: 非匹配边加入, 匹配边删除

=> $|M|$ 增加 1

这解释了为什么增广路的两端必须是未匹配点：如果起点已经被匹配，那么新增第一条边会冲突；如果终点已经被匹配，最后也只是把冲突传递下去，而没有真正释放出一个 +1 的位置。内部点则必须是已匹配点，因为它们在路径中承担“被前一条非匹配边占用，再通过原来的匹配边释放另一端”的传递作用。

路径位置	点的状态	作用
起点	未匹配左点	提供初始空位，使第一条非匹配边可以加入
内部右点	已匹配右点	通过其唯一 matching 边传回左点
内部左点	已匹配左点	从这里继续走非匹配边，尝试替代/传递
终点	未匹配右点	终止传递，完成净 +1

4. 第一版暴力：枚举所有交替简单路径

最直接的构造是：从一个未匹配左点出发，沿着非匹配边、匹配边、非匹配边、匹配边交替走，直到碰到一个未匹配右点。为了避免环，可以要求路径中的点不重复。

这在正确性上很自然，但复杂度立刻爆炸。原因是每次从左点走非匹配边时，可能有很多右点可选；每个已匹配右点又确定地传回一个左点；然后再次分叉。路径内不重复只能避免无限循环，不能避免不同简单路径的数量指数级增长。

暴力对象：所有交替简单路径
去环条件：路径内部不重复点
问题：简单路径数量本身最坏仍可指数级

这一步很像在 NP-hard 问题里直接枚举证书：不是证书定义错了，而是证书空间太大。你能找到证书就能改进，但不能真的把所有证书都枚举出来。

这里的关键失败
“路径”作为搜索对象太细。路径包含完整历史，而完整历史很可能不是未来扩展所必需的信息。

5. 失败路线：按边集合、matched-edge 子集、最后一条边去想

接下来很自然会尝试压缩：既然整条路径太细，那么能不能只看路径里用到了哪些匹配边？翻转后被删除的是 matched edges，被新增的是 non-matched edges。似乎只要知道哪些 matched edge 被替换，就能判断 +1。

这个想法比“路径排列”粗，但仍然不够。因为增广路不仅要选若干 matched edges，还必须能用原图中的 non-match 边把它们按正确顺序串成一条连续交替路径。只知道集合，不知道连接顺序和端点状态，合法性会丢失；如果把顺序也放回状态，又退回路径枚举。

另一个尝试是枚举最后一条边。最后一条边一定是 non-match edge，右端是未匹配右点，左端是已匹配左点。左端的 matched partner 唯一，所以倒数第二条 matched edge 也确定。这看起来能减少搜索。

但倒数第三条 non-match edge 又开始分叉。也就是说，match 边提供唯一性，non-match 边提供分叉。只靠固定最后一条边，不能消除前缀的组合爆炸。

尝试	为什么有吸引力	为什么还不够
枚举路径	最直接，合法性完整	指数级简单路径
枚举 matched-edge 子集	比路径顺序更粗	丢掉连接关系和方向
枚举最后一条 non-match 边	贴近终点条件，倒数第二条边唯一	前缀仍然有大量 non-match 分叉
DFS 不回溯	看起来利用 match 边唯一性	non-match 分叉处选错会漏解

6. 真正的转向：不是选一条路径，而是构造可达点集合

反直觉的地方在这里：看到分叉很多，人容易想“应该 DFS 深挖一条路，或者贪心选一个分支”。但真正能降复杂度的方向恰好相反：不要选分支，而是把一批在相同状态下可达的点合并成集合。

这就是路径历史的等价关系：如果两条不同的交替前缀最终到达同一个左点，并且下一步都需要走 non-match 边，那么从未来扩展的角度看，它们等价。后续只关心“这个左点已经可达”，不关心它是从哪条具体路径来的。

路径历史太细：
L0 - R1 - L1 - R3 - L3
L4 - R2 - L1 - R3 - L3

若二者最终都让 L3 处于“下一步走 non-match 边”的状态，
则后续扩展只需要保留：L3 已可达。

核心等价关系
两个历史路径如果到达同一个点，并且到达后的下一步合法边类型相同，那么未来搜索能力相同，可以合并。

这一步就是从“枚举路径”变成“枚举状态”。状态不再是一条完整路径，而是一个点是否已经在正确的交替层次中被到达。

7. 初始集合：所有未匹配左点，而不是一个虚拟大杂烩

如果目标是找从未匹配左点出发的增广路，那么初始层就应当是所有当前未匹配的左点。这个选择比“虚拟点连所有左点”更精确，因为它保留了“起点必须提供空位”的结构。

L_reached = { 所有当前未匹配的左点 }
R_reached = 空集合

这可以理解成多源 BFS：不是枚举每个未匹配左点单独搜索，而是把所有合法起点同时放进同一层可达集合。这样做的意义不是“偷懒”，而是为了合并状态：不同起点如果后来到达同一个左点，就不需要继续区分来源。

第一步从 $L_reached$ 通过原图边扩展到右点。由于这些左点都未匹配，所以第一条边不可能是 matching 边。扩展时先检查是否能直接到达未匹配右点；如果能，长度 1 的增广路已经找到，可以立即翻转。

8. 集合扩展规则：non-match 扩右点，match 边传回左点

如果第一步没有碰到未匹配右点，那么被扩展到的右点必须是已匹配右点。每个已匹配右点在当前 matching 中都有唯一对应的左点，因此可以沿 matching 边确定地传回左侧。

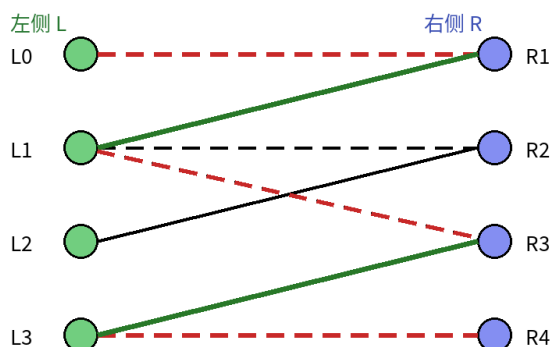
当前有一批可达左点 $L_reached$ 。

1. 从它们沿 non-match 边扩展，得到右点候选。
2. 如果其中有未匹配右点：找到增广路。
3. 如果右点已匹配：沿唯一 matching 边传回对应左点。
4. 新左点加入 $L_reached$ ，继续。

这里 match 边的作用非常关键：它不是产生组合爆炸的地方，而是压缩的地方。一个 matched right 只对应一个 matched left，所以右点到左点的回传没有选择。组合爆炸来自左点沿 non-match 边扩右点，但这些右点被加入集合后会被去重，不再作为路径组合枚举。

因此算法不是“所有路径一起枚举”，而是“所有可能到达的点一起推进”。这就是为什么最终形式看起来像 BFS，但它不是一开始那个盲目 BFS，而是在理解了等价关系之后得到的 BFS。

交替可达集合：不是枚举路径，而是推进状态集合



图例

实线 = 当前 matching 边
虚线 = 非 matching 边
红/绿 = 一条增广路

点集视角

只维护已到达的左点集合；
完整路径历史可以忘掉。
已匹配右点通过 matching
唯一传回对应左点。

9. 不变量：为什么可以忘掉路径历史

要让集合扩展成为正确算法，必须有不变量支撑。一个自然的不变量是：

每个进入 $L_reached$ 的左点，都存在一条从某个未匹配左点出发的交替路径到达它，并且这条路径最后一条边是 matching 边，或者它就是初始未匹配左点。
因此从这个左点继续扩展时，下一条边应当是 non-match 边。

对应地，每个进入 $R_reached$ 的右点，都是从某个 $L_reached$ 左点沿 non-match 边到达的。如果该右点未匹配，则这条路径已经是增广路；如果该右点已匹配，则它可以通过唯一的 matching 边传回一个左点。

这个不变量解释了为什么只存点集合足够。到达同一个左点的不同路径，虽然历史不同，但它们都满足同一个状态：下一步该走 non-match 边。未来可扩展的边只由当前左点和当前 matching 决定，而不由之前路径决定。

把手

真正的“把手”是：在交替路径的正确层次上，一个点的可达性足以代表所有到达它的历史。

10. 终止条件：没有新点加入，为什么就找不到增广路？

每一轮扩展都会尝试把新的右点或新的左点加入集合。如果某一轮没有任何新点加入，说明从当前所有已知可达左点出发，已经无法通过合法的交替规则到达更多状态。这个交替可达闭包已经稳定。

如果在闭包稳定之前碰到未匹配右点，就找到了增广路。如果闭包稳定后仍然没有碰到未匹配右点，则不存在从任何未匹配左点出发的交替路径到达未匹配右点，也就不存在增广路。

若搜索过程中找到 unmatched right：成功，翻转路径， $|M| + 1$ 。

若没有新点可加入且从未碰到 unmatched right：失败，当前 M 无增广路。

更深一层，Berge 定理告诉我们：一个 matching 是最大 matching，当且仅当不存在增广路。因此，如果这个搜索找不到增广路，当前 matching 就已经最大。

11. 为什么这不是“DFS 不回溯”

在推导中曾经出现一个想法：既然 matched edge 唯一，那么 DFS 是否可以不用回溯？问题在于，唯一性只发生在从 matched right 沿 matching edge 传回 left 的那一步。真正产生分叉的是从 left 沿 non-match edge 扩展到 right 的那一步。

如果 DFS 在 non-match 分叉处不回溯，选错一个右点就可能走到死路，而另一个右点才通向增广路。枚举所有第一条边也不够，因为第三条、第五条、后续每个 non-match 层都可能再次分叉。

BFS/集合构造的优势不是“比 DFS 更神奇”，而是它不把分叉理解为路径选择，而是把所有分叉产生的右点加入集合，然后通过 matched edge 统一传回左点集合。

12. 复杂度：从指数路径到多项式状态

暴力枚举路径的复杂度是“交替简单路径数量”，最坏指数级。状态集合版本只会把每个点加入集合有限次。每条边在扩展时被检查有限次。因此一次寻找增广路的过程可以做到 $O(E)$ 级别，或者在朴素实现中是多项式。

如果每次只找一条增广路并翻转，最多翻转 $O(V)$ 次，所以朴素增广路算法整体是多项式。更进一步的 Hopcroft-Karp 会一次找多条最短增广路，把复杂度进一步降到 $O(E \sqrt{V})$ ，但那是另一个层次的优化。当前这份推导的价值在于理解为什么“单条增广路搜索”本身可以从指数压到多项式。

版本	搜索对象	复杂度直觉
暴力路径枚举	所有交替简单路径	指数级
边集合/子集枚举	matched-edge 子集或路径边集	仍可能指数级
点集合可达性	左点/右点的交替可达状态	多项式

13. 这次推导的思维价值

这次最有价值的不是“知道了二分图匹配算法”，而是重走了一个算法发现过程。这个过程可以迁移到许多未知问题：

- 先把优化目标拆成最小改进单位，问是否存在 +1 证书。
- 先允许暴力，明确暴力枚举的对象是什么。
- 定位组合爆炸来自哪里：路径、边集合、点集合还是分支历史。
- 寻找等价关系：哪些历史对未来扩展没有影响？
- 用不变量支撑状态合并，而不是凭直觉合并。
- 把“枚举证书”改造成“构造可达闭包”。

这也是为什么它会让人联想到 FPT 或 NP-hard 问题的思维训练：很多困难问题也会先变成“给定参数/目标，是否存在证书”的形式，然后试图通过剪枝、kernel、状态压缩、分支规则来控制组合爆炸。区别在于，二分图 matching 的结构足够强，最终能把证书搜索完全压成多项式的交替可达集合。

最终总结
算法表面是 BFS；发现过程的内核是：+1 可行性证书 + 组合爆炸定位 + 状态等价关系 + 交替可达闭包。

14. 用一句话重述最终算法

从所有未匹配左点出发，维护交替可达的左点集合；
反复沿非匹配边扩展到右点；
若到达未匹配右点，则得到增广路并翻转；
若到达已匹配右点，则沿其唯一匹配边回到左点并加入左点集合；
若没有新点可加入，则不存在增广路。

这句话已经是算法。但真正重要的是，前面的推导解释了为什么它不是“随便 BFS 一下”，而是把指数级路径历史压缩成多项式点状态之后的 BFS。